Babystars and soil salinity

Anthocyanins in Babystars

Anthocyanins are a group of pigments that contribute to petal color intensity across many species of flowers. These pigments also are important for providing tolerance to abiotic stressors, such as high UV radiation, extreme soil conditions, and low precipitation.

Leptosiphon minimus (common name: True babystars) is a native plant species found across the northwestern coast of North America in California, Oregon, Washington, and British Columbia. *L. minimus* is known for being polymorphic, exhibiting two petal colors: pink and white. Within these color morphs, elevated levels of anthocyanins produce a pink pigment.

In our project, we are interested in assessing if certain abiotic stressors, specifically harsh, more saline soils, can predict the frequency of pink flowers for *L. minimus* at the population level. We will be using iNaturalist observations taken across the northwestern coast of USA and Canada to identify the pink vs. white color morphs of *L. minimus* and group them by different populations. We will also use distance to shoreline as a proxy for soil salinity.

Question: Are *L. minimus* flowers more likely to be pink the closer they are located to the shore?

Hypothesis: Soils closer to the sea shore have higher salinity, and increasing anthocyanin levels can help plants better withstand exposure to more saline microenvironments.

Prediction: If anthocyanins help plants better withstand exposure to saline soils, then we predict that *L. minimus* flowers closer to the sea shore will be more likely to be pink.

Set up Working Directory and Install/Library Packages

Let's start by loading the required packages (and installing if necessary).

```r
#install.packages("dbscan")
#install.packages("sf")
#install.packages(("ggspatial"))
#install.packages("rnaturalearthdata")
#install.packages("scatterpie")
#install.packages("rnaturalearth")
#install.packages("sf")
#install.packages("tmap")

library(scatterpie)
library(dbscan)
library(sf)
library(dplyr)
library(tidyr)
library(ggplot2)
library(ggspatial)
library(rnaturalearth)
library(tmap) # To visualize shoreline shape files
library(brms) # For Bayesian Statistical Modeling
library(ggeffects)
```

Import data set into R

We have downloaded 69 observations of *Leptosiphin minimus* (babystars) from iNaturalist as a CSV file (called `Lmini_iNat.csv`). This will be our raw data, and includes the following columns: the observation ID (`id`), the URL to the observation on iNaturalist.com (`url`), latitude/longitude for the observation location (`latitude` and `longitude`) and the scientific name.

```r
#Read in CSV
LM <- read.csv("data/LMini_iNat.csv")

#Check out names of columns
names(LM)
```

```
[1] "id"              "url"             "latitude"        "longitude"
[5] "scientific_name"
```

Convert from Degrees to Meters

In order to define populations, we will use a package called `dbscan` to group *L. minimus* observations into populations using a 1000-meter radius. Before we can do this, we must convert latitude/longitude degrees into meters.

```
#Convert to sf object
LM_sf <- st_as_sf(LM, coords = c("longitude", "latitude"), crs = 4326)

#Project to meters
LM_utm <- st_transform(LM_sf, 32611)  # change UTM zone if needed
LM_coords <- st_coordinates(LM_utm)
```

Run `dbscan` and add define populations

Now that we've converted latitude to meters, we can run `dbscan`. We will use `dbscan` to define populations as 4 or more observations within a 1000 m radius.

```
#Run DBSCAN, 1000 m radius populations with 4 or more observations per pop
db <- dbscan(LM_coords, eps = 1000, minPts = 4)

#Add population column
LM$population_id <- db$cluster
```

Save data frame with population ID

We will save the data frame with the new population ID column as a new CSV file called `LMini_iNat_Pops.csv`.

```
write.csv(LM, "data/LMini_iNat_Pops.csv", row.names = FALSE)
```

Visualize data

Let's count how many observations are within each of populations 0-6. **NOTE: All observations belonging to population '0' are singleton observations, which did not meet our population criteria (4 observations within a 1000 m radius).**

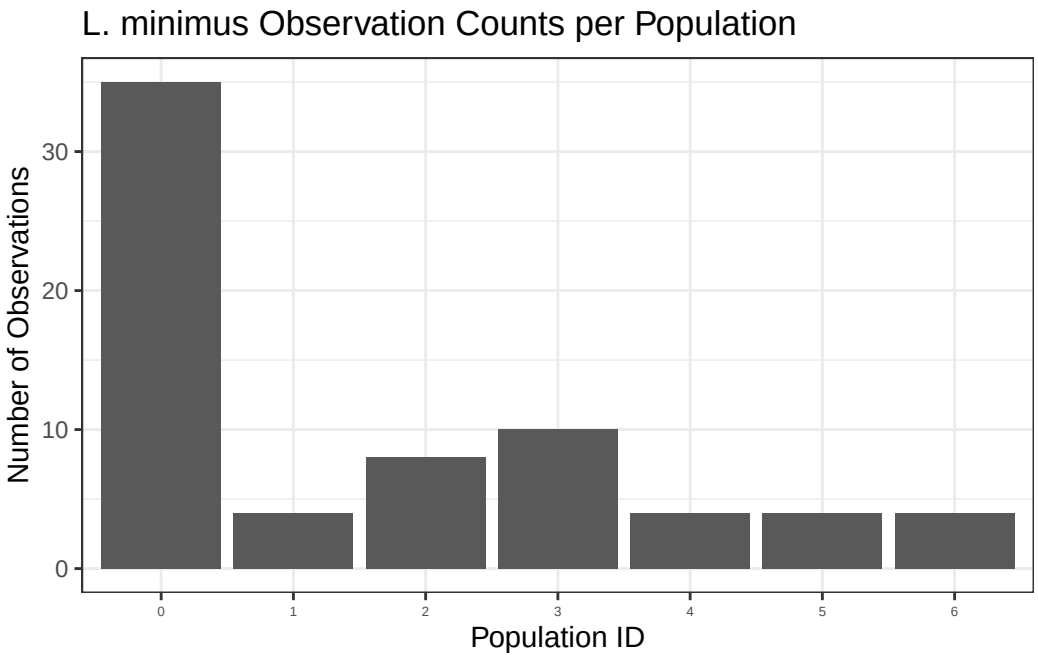
```
#List pop sizes
LM_pop_sizes <- LM %>%
  count(population_id, name = "n_obs") %>%
  arrange(desc(n_obs))

LM_pop_sizes
```

```
  population_id n_obs
1             0    35
2             3    10
3             2     8
4             1     4
5             4     4
6             5     4
7             6     4
```

```
# Count up the total number of observations that are NOT in population "0"
sum(LM$population_id == 0)
```

```
[1] 35
```

```
# View a bar plot of population sizes
ggplot(LM_pop_sizes, aes(x = factor(population_id), y = n_obs)) +
  geom_col() +
  labs(x = "Population ID",
       y = "Number of Observations",
       title = "L. minimus Observation Counts per Population") +
  theme_bw() +
  theme(axis.text.x = element_text(size = 5))
```

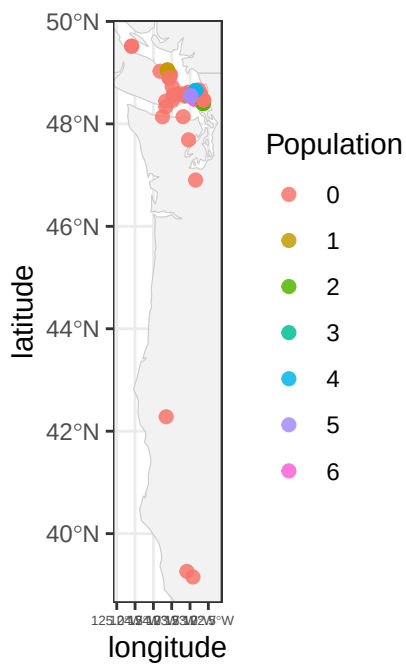


Visualize populations on a map

Let's visualize all observations on a map of the West Coast of North America, and color the points by population. Remember that population = 0 are singletons.

```
# Get world basemap
world <- ne_countries(scale = "medium", returnclass = "sf")

# Add a bounding box around our data
bbox <- c(
  xmin = min(LM$longitude) - 0.5,
  xmax = max(LM$longitude) + 0.5,
  ymin = min(LM$latitude) - 0.5,
  ymax = max(LM$latitude) + 0.5
)

# Plot
ggplot() +
  geom_sf(data = world, fill = "gray95", color = "gray80") +
  geom_point(data = LM,
```

```
             aes(x = longitude, y = latitude,
                 color = factor(population_id)),
             size = 2, alpha = 0.85) +
  coord_sf(xlim = c(bbox["xmin"], bbox["xmax"]),
           ylim = c(bbox["ymin"], bbox["ymax"]),
           expand = FALSE) +
  labs(color = "Population") +
  theme_bw() +
  theme(axis.text.x = element_text(size = 5))
```



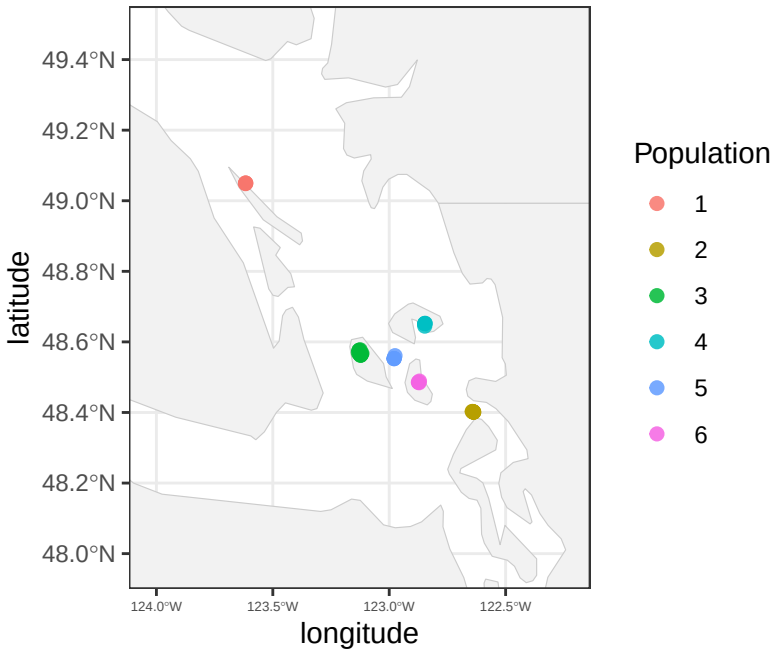
Lets remove noise from 'pop 0' singleton observation

Remove single observations by filtering for observations where population ID is *not* 0.

```
# Remove noise points (population_id == 0)
LM_clean <- LM %>%
  filter(population_id != 0)

# Quick check
table(LM_clean$population_id)
```

```
 1  2  3  4  5  6
 4  8 10  4  4  4
```

```r
# Bounding box around our data
bbox <- c(
  xmin = min(LM_clean$longitude) - 0.5,
  xmax = max(LM_clean$longitude) + 0.5,
  ymin = min(LM_clean$latitude) - 0.5,
  ymax = max(LM_clean$latitude) + 0.5
)

# Plot cleaned data on a new map
ggplot() +
  geom_sf(data = world, fill = "gray95", color = "gray80") +
  geom_point(data = LM_clean,
             aes(x = longitude, y = latitude,
                 color = factor(population_id)),
             size = 2, alpha = 0.85) +
  coord_sf(xlim = c(bbox["xmin"], bbox["xmax"]),
           ylim = c(bbox["ymin"], bbox["ymax"]),
           expand = FALSE) +
  labs(color = "Population") +
  theme_bw() +
   theme(axis.text.x = element_text(size = 5))
```



Save data frame with population ID + singletons removed

We will save the data frame after filtering out singletons as new CSV file called `LMini_iNat_Pops_filtered.csv`.

```
# Save to new CSV
write.csv(LM_clean, "data/LMini_iNat_Pops_filtered.csv", row.names = FALSE)
```

Sorting pink and white flowers

Now that we have our data set, we need to mark iNaturalist observations based on flower color

Lets make a copy of **`LMini_iNat_Pops_filtered.csv`** called **`LMini_iNat_Pops_filtered_curated.csv`**, and add some new columns: **`polymorphic`** and **`color1`** and **`color2`**.

```
# Import filtered data frame from last step
LMini_iNat_Pops_filtered = read.csv("data/LMini_iNat_Pops_filtered.csv")

# Add polymorphic, color1, and color2 columns
LMini_iNat_Pops_filtered_curated <- LMini_iNat_Pops_filtered %>%
  mutate(
    polymorphic = NA, # Set to NA to start
    color1 = NA_character_,
    color2 = NA_character_
  )

# Export the CSV file - important, because next step is done manually!
write.csv(LMini_iNat_Pops_filtered_curated,
          "data/LMini_iNat_Pops_filtered_curated.csv",
          row.names = FALSE)
```

Next, we will go through each iNaturalist observation in our LMini_iNat_Pops_filtered_curated.csv by clicking the hyperlink in each respective row. Then we will manually note the flower color in the observation's photo, choosing **pink** or **white**, and writing that down in the cell of the color column for each respective row. However, if the observation has no flowers or all flowers are still closed, we will instead write **closed** in that cell instead. Additionally, we will mark whether the observation show more than one flower color in the polymorphic column. This will be a binary variable, with **polymorphic observations receiving a 1** and **monomorphic observations receiving a 0.** If the observation is polymorphic, it's second color will be put into the color2 column.

Once the **`polymorphic`** and **`color`** columns are filled out we can move on to the next step; lets save the new file as **`LMini_iNat_Pops_filtered_curated_color.csv`** Make sure to save it as a CSV after editing.

Filtering out closed flowers

Read in the CSV from the last step, and filter out the closed flowers.

```r
#Read in curated CSV
LMC <- read.csv("data/LMini_iNat_Pops_filtered_curated_color.csv")

LMCF <- LMC %>%
  filter(!(color1 == "closed" & color2 == "closed"))
```

```r
#Combine remaining color columns into one column and remove empty rows
LMCF_clean <- LMCF %>%
  pivot_longer(
    cols = starts_with("color"),
    values_to = "color",
    names_to = "slot"
  ) %>%
  filter(
    !is.na(color),
    color != "",
    color != "closed"
  )
```

Group colors by population

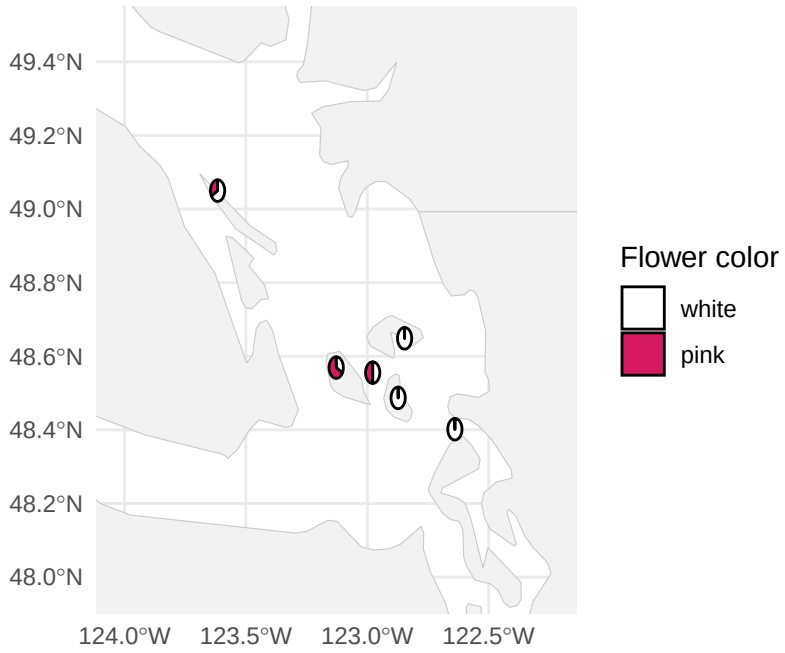
To summarize the proportion of flowers in each population that are pink vs. white, we will summarise each population into a single point by averaging the latitude and longitude. We will make a new data frame `pop_pies` that summarizes the average latitude/longitude, the number of white flowers, the number of pink flowers, and the number of total flowers per population.

```r
#Grouping colors by population for pie charts
pop_pies <- LMCF_clean %>%
  group_by(population_id) %>%
  summarise(
    latitude  = mean(latitude),
    longitude = mean(longitude),
    white = sum(color == "white"),
    pink  = sum(color == "pink"),
    total_n = n(),
    .groups = "drop"
  )
```

Mapping populations as pie charts

Let's make a new map that shows a small pie chart for each population, using the average lat/lon for each as its markers, and denoting the proportion of all flowers that are pink vs. white.

```
# World basemap
world <- ne_countries(scale = "medium", returnclass = "sf")

# Bounding box from population-level data
bbox <- c(
  xmin = min(pop_pies$longitude) - 0.5,
  xmax = max(pop_pies$longitude) + 0.5,
  ymin = min(pop_pies$latitude) - 0.5,
  ymax = max(pop_pies$latitude) + 0.5
)

# Plot map with pie charts
ggplot() +
  geom_sf(data = world, fill = "gray95", color = "gray80") +
  geom_scatterpie(
    data = pop_pies,
    aes(x = longitude, y = latitude),
    cols = c("white", "pink"),
    pie_scale = 1.5,
    color = "black"
  ) +
  coord_sf(
    xlim = c(bbox["xmin"], bbox["xmax"]),
    ylim = c(bbox["ymin"], bbox["ymax"]),
    expand = FALSE
  ) +
  scale_fill_manual(
    values = c(white = "white", pink = "#d81b60"),
    name = "Flower color"
  ) +
  theme_minimal()
```



Mapping distance from shore

In order to calculate the distance of each observation from the water (i.e., to the closest shore), we will compute the minimum distance from each observation (point) to the shoreline using vectors from the NOAA National Shoreline CUSP (West region) dataset. We downloaded the West region shape file from the NOAA portal (https://nsde.ngs.noaa.gov/), and imported this into R. **Note: This only works for observations in the US, so we have to manually calculate the distance from shore for the Canadian observations (population 1) using Avenza.**

```
# Import shape file of Western US shoreline
shoreline <- st_read("data/West/West.shp")
```

```
Reading layer `West' from data source
  `/Users/agrearson/Documents/3_UCSC/2_Year2/BIOE176 - Data science/Data_Sci_Leptosiphon_Poly
  using driver `ESRI Shapefile'
Simple feature collection with 82945 features and 14 fields
Geometry type: LINESTRING
Dimension:     XY
Bounding box:  xmin: -124.7849 ymin: 32.53445 xmax: -116.9286 ymax: 49.00209
Geodetic CRS:  NAD83
```

```r
# To view the imported shape file - Requires HTML output for saving; commented out for now
#tmap_mode("view")  # interactive mode
#tm_shape(shoreline) + tm_lines()

# Import coordinate data for observations
points_sf <- st_as_sf(LMCF_clean,
                      coords = c("longitude", "latitude"),
                      crs = 4326)

# Project to a suitable projection
shoreline_proj <- st_transform(shoreline, 32610) # UTM 10N
points_proj    <- st_transform(points_sf, 32610)

# Compute minimum distances to shore - add new column called dist_m
LMCF_clean$dist_m <- apply(st_distance(points_proj, shoreline_proj), 1, min)

# Manually overwrite data for observations in Canada (Population 1)
LMCF_clean$dist_m[6] <- 200 # Observation 121793064
LMCF_clean$dist_m[7] <- 210 # Observation 121793096
LMCF_clean$dist_m[8] <- 190 # Observation 121793133

# Change pink to 1 and white to 0, add a new column called color_binom
LMCF_clean <- LMCF_clean %>%
  mutate(color_binom = case_when(
    color == "pink"  ~ 1,
    color == "white" ~ 0,
    TRUE ~ NA_real_
  ))

# Check headers for data frame
str(LMCF_clean)
```

```
tibble [22 x 12] (S3: tbl_df/tbl/data.frame)
 $ scientific_name: chr [1:22] "Leptosiphon minimus" "Leptosiphon minimus" "Leptosiphon mini
 $ url            : chr [1:22] "https://www.inaturalist.org/observations/27733914" "https://
 $ latitude       : num [1:22] 48.4 48.6 48.6 48.4 48.4 ...
 $ longitude      : num [1:22] -123 -123 -123 -123 -123 ...
 $ hyperlink      : chr [1:22] "https://www.inaturalist.org/observations/27733914" "https://
 $ population_id  : int [1:22] 2 3 3 2 2 1 1 1 4 3 ...
 $ polymorphic    : chr [1:22] "0" "0" "0" "0" ...
 $ notes          : chr [1:22] "" "" "" "" ...
 $ slot           : chr [1:22] "color1" "color1" "color1" "color1" ...
```

```
 $ color          : chr [1:22] "white" "pink" "pink" "white" ...
 $ dist_m         : num [1:22] 280 1350 1518 212 289 ...
 $ color_binom    : num [1:22] 0 1 1 0 0 1 0 0 0 0 ...
```

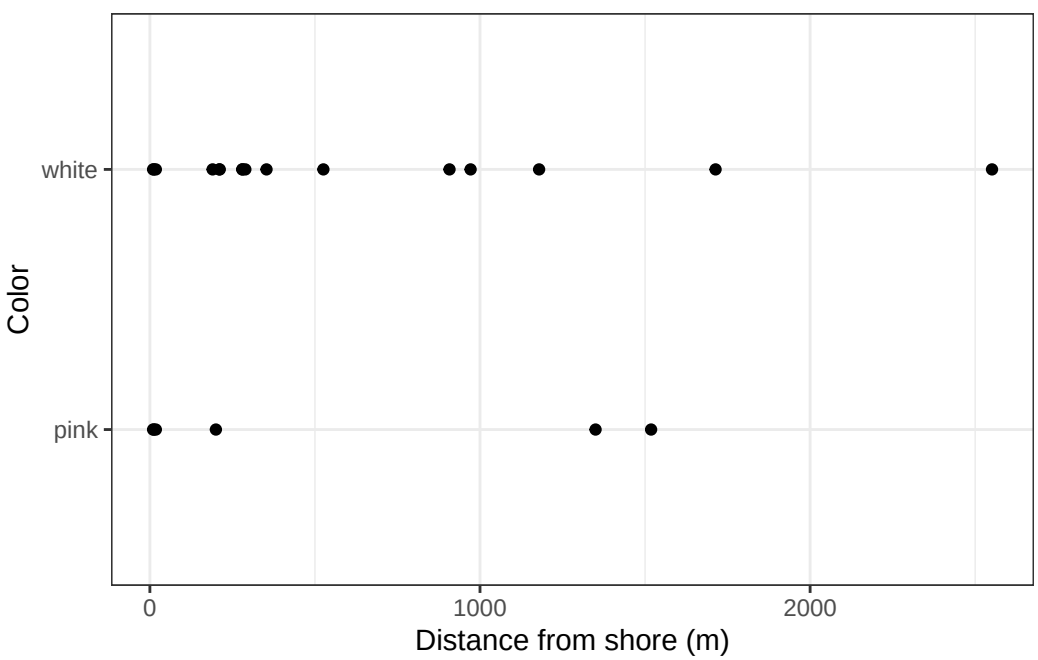
We will export the data frame after calculating the distance from shore for each observation as `LMCF_clean_dist.csv`.

```
# Write to csv
write.csv(LMCF_clean,"data/LMCF_clean_dist.csv", row.names = FALSE)
```

Make a simple plot for our potential model

To quickly assess how probability of being pink might change with distance from shore, let's create a simple scatter plot of color vs. distance (in meters).

```
# Plot distance from shore vs. color
ggplot(data=LMCF_clean, aes(x=dist_m, y=color)) +
  geom_point() +
  theme_bw() +
  labs(x = "Distance from shore (m)",
       y = "Color")
```



Run prior predictive simulations

In this section, we will define our priors and explore what they look like in simulations.

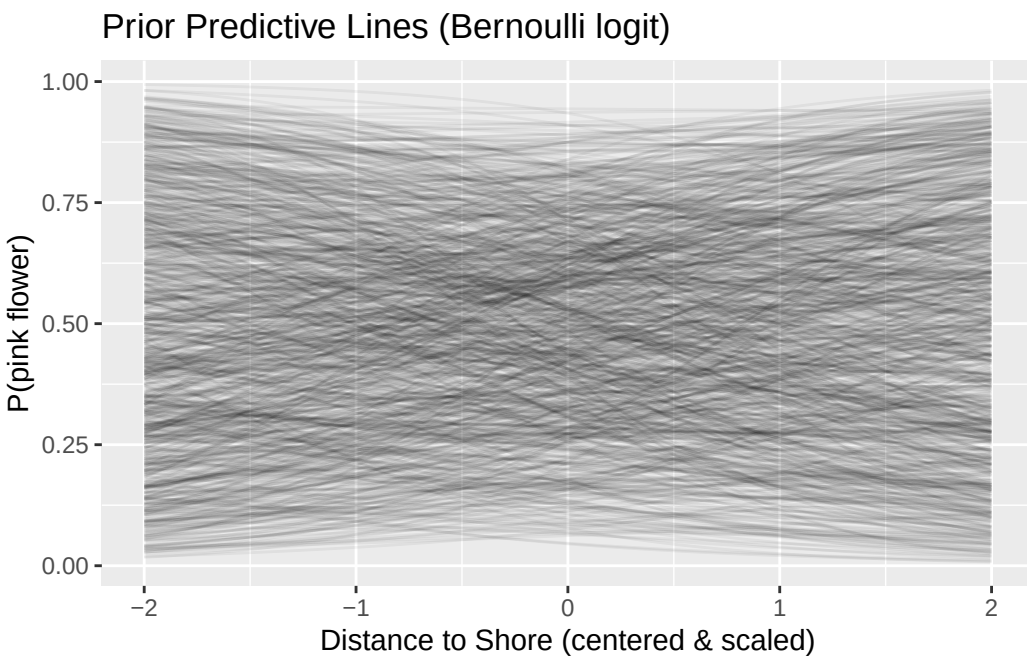
For our standard deviation, we will set our prior as exponential(1). This will make our standard deviation always positive and centreed near 0.

For our slope, we will set it as normal(0, 0.5). Our slope is how much the log-odds of being pink changes for every one unit increase in distance to shore. The 0 means we start with no prior expectation of a relationship, and the 0.5 means we're skeptical of very large effects.

Finally, for our intercept, we will set it as normal(0,1). The intercept is the baseline log-odds of a flower being pink when distance to shore is zero. The 0 means we expect the baseline probability to be centered around 50/50 (since plogis(0) = 0.5). The 1 means we allow moderate uncertainty around that, letting the baseline probability range roughly between ~18% and ~82% (plogis(-1) to plogis(1)).

```
# Define priors
priors <- c(
  set_prior("normal(0, 1)", class = "Intercept"),        # prior for alpha (intercept)
  set_prior("normal(0, .5)", class = "b", coef = "dist_m"), # prior for beta (slope)
  set_prior("exponential(1)", class = "sd")                 # prior for random effect SD
)

# Run prior simulations
n <- 1000

priorsims <- tibble(
    group = seq_len(n),
    alpha = rnorm(n, 0, 1),   # prior for intercept (log-odds scale)
    beta  = rnorm(n, 0, .5)      # prior for slope (effect of distance)
  ) %>%
  expand(
    nesting(group, alpha, beta),
    dist_scaled = seq(from = -2, to = 2, length.out = 100)  # range of (scaled) distance
  ) %>%
  mutate(
    log_odds = alpha + beta * dist_scaled,   # linear predictor (log-odds)
    prob      = plogis(log_odds)             # convert to probability via inverse logit
  )

# Plot prior predictive lines
ggplot(priorsims,
       aes(x = dist_scaled,
```

```
          y = prob,
          group = group)) +
  geom_line(alpha = 0.05) +
  labs(
    title = "Prior Predictive Lines (Bernoulli logit)",
    x     = "Distance to Shore (centered & scaled)",
    y     = "P(pink flower)"
  )
```



Run the model with our priors

We will fit a GLMM model to ask whether the probability of being pink changes with distance to shore, specifically using a logistic mixed effects model. Because the response variable (i.e., chance of being pink, where pink = 1 and white = 0) is binary, we will fit a Bernoulli distribution with a logit link. So, we are modeling the log-odds of being pink (i.e., `color_binom` = 1). We are modeling whether distance predicts the probability of being pink, with the population as a random effect.

```
# Run the model
m.dist.color.pop <-
  brm(data = LMCF_clean, # Give the model the data
```

```r
      # Use a binomial distribution
      family = bernoulli(link = "logit"),
      # Specify the model
      color_binom ~ 1 + dist_m + (1|population_id), # Add population as a random effect
      # Set priors
      prior = priors,
      sample_prior = TRUE,
      iter = 2000, warmup = 1000, chains = 4, cores = 4,
      file = "output/m.dist.color.pop")
```

Assess the model

Let's run `summary()`, `print()`, and `plot()` on our model output to assess how well it ran!

The model seems to have converged correctly! Our Rhat is equal to 1. The MCMC chains are overlapping and flat, and the posterior distributions are smooth and have one clean peak.

For every 1 meter increase in distance to shore, the log-odds of being pink increases by 0.003. However, we cannot confidently state that the effect size is different from 0 because our 95% lower and upper compatibility intervals intersect 0 (-0.001 - 0.010).

```r
summary(m.dist.color.pop)
```

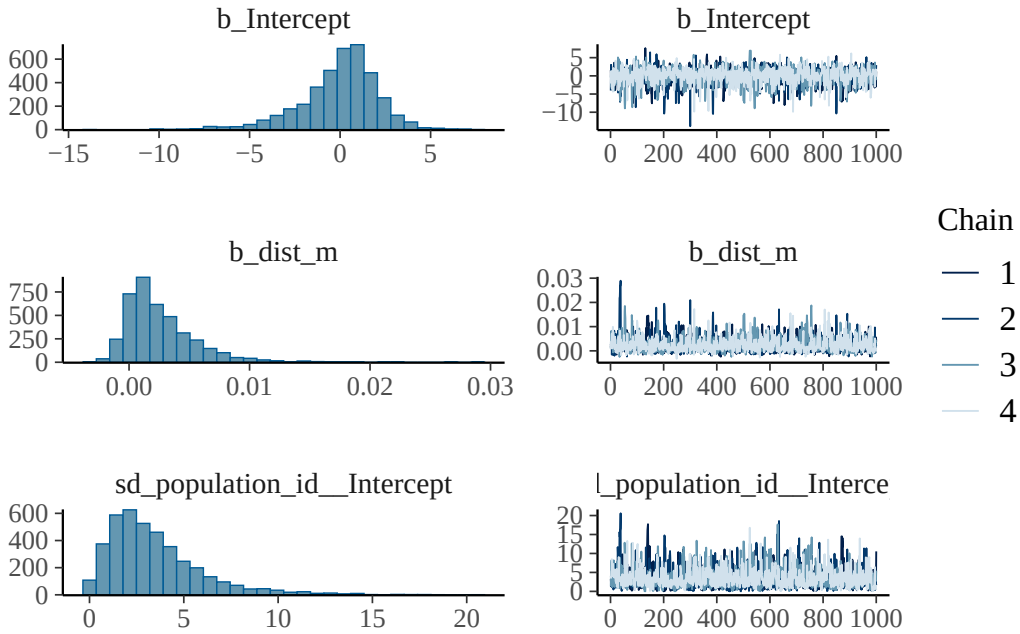
```
 Family: bernoulli
  Links: mu = logit
Formula: color ~ 1 + dist_m + (1 | population_id)
   Data: LMCF_clean (Number of observations: 22)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Multilevel Hyperparameters:
~population_id (Number of levels: 6)
              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sd(Intercept)     3.46      2.53     0.34    10.14 1.00      832     1264

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept    -0.03      2.18    -5.29     3.58 1.00     1524     1535
dist_m        0.00      0.00    -0.00     0.01 1.00     1391     1354

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
```

```
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
print(m.dist.color.pop, digits = 3)
```

```
 Family: bernoulli
  Links: mu = logit
Formula: color ~ 1 + dist_m + (1 | population_id)
   Data: LMCF_clean (Number of observations: 22)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Multilevel Hyperparameters:
~population_id (Number of levels: 6)
              Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS Tail_ESS
sd(Intercept)    3.462     2.529    0.337   10.137 1.002      832     1264

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS Tail_ESS
Intercept   -0.033     2.180   -5.289    3.582 1.002     1524     1535
dist_m       0.003     0.003   -0.001    0.010 1.002     1391     1354

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
plot(m.dist.color.pop)
```



How often is our posterior slope larger than zero?

In this section, we will use the `as_draws_df()` to extract the posterior distribution samples from our fitted `brms` model into a data frame. Then, we will calculate a proportion of how many our sampled values of our slope are positive.

```
#  Extract all posteriors from 4 MCMC chains into a table
draws <- as_draws_df(m.dist.color.pop)

# What proportion of posteriors have estimate for slope > 0?
sum(draws$b_dist_m>0)/length(draws$b_dist_m)
```

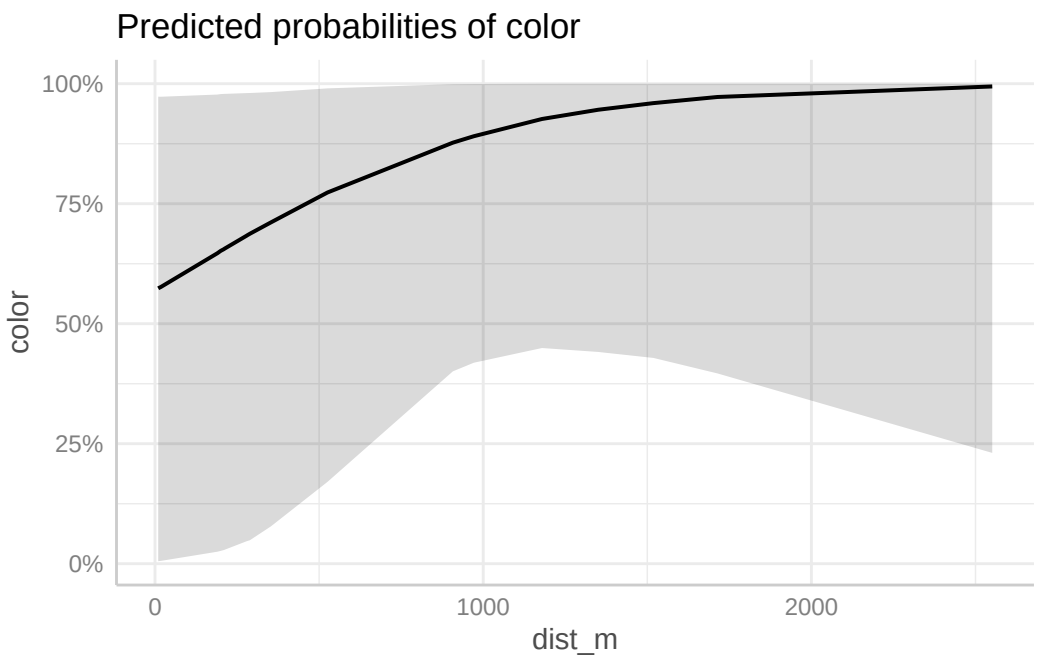
```
[1] 0.858
```

We have a value of .858, which means that **85.8% of the posterior distribution for the slope is above zero.** This means that there is an 85.8% probability that distance to shore has a positive effect on pink flower color. In other words, flowers are more likely to be pink as distance from shore increases.

```
# Plot predicted response
preds <- predict_response(m.dist.color.pop,
                          bias_correction = TRUE)
```

```
Problem with these data entries:
yobs
   4

Problem with these data entries:
yobs
   4
```

```r
plot(preds)
```

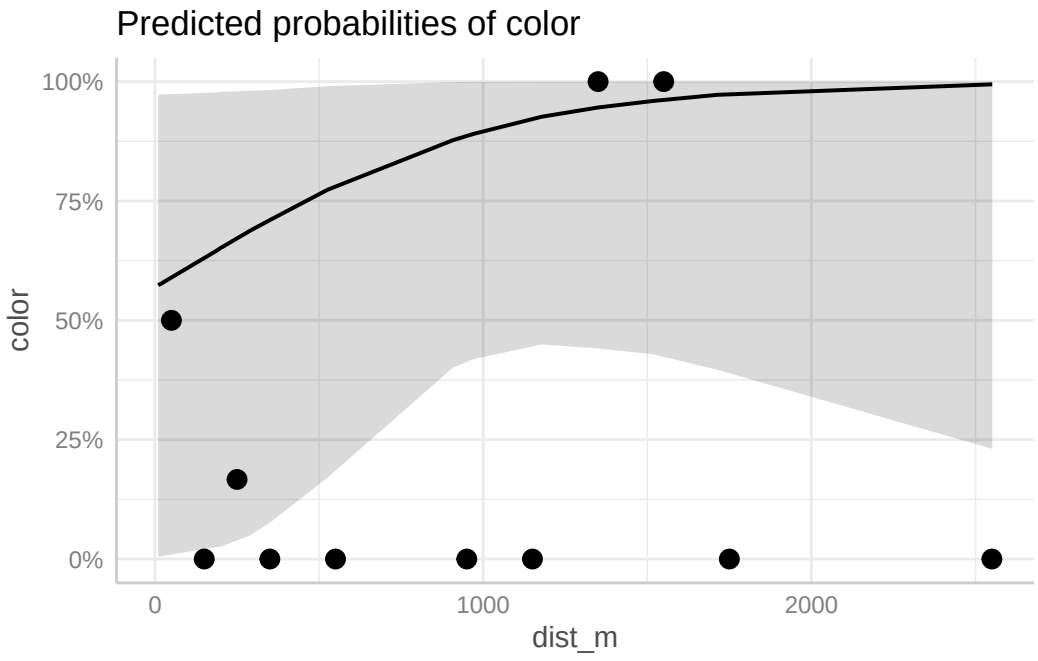


Overlay empirical binned proportions on top of model

To better illustrate the trend, we will overlay the empirical data on top of the model predictions in the `predict_response` plot. We will:

1. Bin distance to shore into 250m bins
2. Calculate the observe proportion of flowers that are pink
3. Add those points to the prediction plot

```r
# Bin observations into 100m bins
LMCF_binned <- LMCF_clean %>%
  mutate(dist_bin = floor(dist_m / 100) * 100)

# Calculate observed proportions of pink in each
obs_summary <- LMCF_binned %>%
  group_by(dist_bin) %>%
  summarise(
    mean_prob = mean(color_binom),
    n = n()
  ) %>%
  ungroup()

# Center points on the bin
obs_summary <- obs_summary %>%
  mutate(dist_mid = dist_bin + 50)

p <- plot(preds) # Call predict_response plot
p + # Add binned points
 geom_point(data = obs_summary,
            aes(x = dist_mid, y = mean_prob),
            size = 3,
            color = "black")
```


Predicted probabilities of color
100%
75%
50%
25%
0%
color
0
1000
2000
dist_m